When we move to distributed systems, communication over the network is an additional source of uncertainty: we know that we sent an HTTP request, but if we don't get an answer, we have no way to know to if the message arrived, if the network is broken, if the server is broken, or if it is hanging on a time-consuming task[9].

We therefore have to decide how to handle this uncertainty: first of all, are we going to wait until we get a response (synchronous layout) or send a request and do something else while we wait for an answer, or yet "fire and forget", i.e. send a request and just not wait for any answer from the remote server?

This communication channel is part of a workflow of user interaction, with users waiting for that response, and their patience is usually limited: who would wait ten minutes (or just two!) before hitting reload on the page?

And indeed for an ecommerce page, where the user expects to see a live update within a reasonable time, usually web servers have short timeouts on the calls they receive, meaning they would respond with a 5XX error after a few seconds, usually less than ten.